TRANSLATOR'S NOTE — English translation of the technical proposal submitted to the 2026 Mirae Asset Securities AI Festival (Financial Product Agent track). The competition build used NCP HyperCLOVA X, as the rules required; this repository's English edition runs on Google Gemini.

10th Mirae Asset Securities AI Festival 2026 · Financial Product Agent Track

Technical Proposal

API END-POINT FOR EVALUATION

`http://223.130.161.217/answer`

Task	Financial Product Agent — building an Ontology · Knowledge Base · Agent on financial-product data
Team	**Wolves of Walnut Street**
Members	**Jang Se-heon (장세헌) · Violette Gimenez · Lee Ho-jeong (이호정)**
Submitted	2026-09-06 (same day as the code freeze)
Repository	`https://github.com/miraeasset-aifestival-2026-product/fin-004`
API End-point	`http://223.130.161.217/answer` (Naver Cloud VPC · Ubuntu 24.04 · Docker Compose)
Answer-generation LLM	NCP HyperCLOVA X `HCX-005` (CLOVA Studio OpenAI-compatible endpoint, function calling)
Target data	The organizers' 4 master datasets, 53,375 rows (domestic bonds · domestic ETF/ETN · foreign ETF/ETN · public/private funds)

Contents

01	**Problem Definition**	The structural difficulties of financial-product data, and the six incidents we actually ran into
02	**Proposed Approach**	Specialized data collection & cleansing / ontology engineering / retrieval methodology
03	**System Architecture**	The load → index → retrieve → generate pipeline, and the deployment setup
04	**Key Functional Flow**	Data collection → ontology design → graph construction → retrieval
05	**Expected Impact · Scalability**	Measured performance, points of business application, extending to more product types
06	**Appendix**	Ontology schema · API specification · evaluation reproduction · data limitations

Three things to read first

① **What we built.** A federated retrieval agent in which a single question splits into five paths — RDB · knowledge graph · lexical index · vector index · document store — and merges back into one answer. Because all five paths reference **one and the same ontology**, they can verify each other's answers.

② **What is different.** We use the ontology as **an execution layer that generates schema cards and value validators at startup**. And we made it standard procedure to recount the numbers produced by SQL through the graph and compare the two.

③ **What we measured.** Accuracy 47/54, average 20.6 s, net contribution of the vector leg +3.5 pp, marginal cost of an extra round +11.2 s. **All of these are measured values.** We report unfavorable results as they are, including the fact that the vector leg contributes little (06-D).

Table 0. Sources of the measured figures in this document

Metric	Value	How to reproduce
Accuracy · response time	47/54 (87%) · average 20.6 s	`scripts/eval_truth.py` → `data/eval/truth_results.jsonl`
Knowledge graph size	1,082,874 triples (965,823 asserted + 117,051 inferred)	`kg_asserted` / `kg_inferred` of `GET /health`
Index size	75,089 documents	`GET /health`
Recall by retrieval method	Measured on 1,313 queries	`python scripts/audit_vector.py --sections BC` (no LLM calls · fixed seed)
Routing · tool calls · latency	Full traces of all 54 questions	`think_trace` in `data/eval/truth_results.jsonl`
Tests	194 passing	`python -m pytest -q`

01 Problem Definition

> On the surface, the task looks like an **NL2SQL** problem: four structured master datasets are given, and natural-language questions must be answered. But once you actually open the data, **"a row is not a product," "the relationship the question asks about does not exist in the data at all," "different kinds (species) are mixed within one table,"** and **"blanks are intended values."** None of these four is solved by writing SQL well. What is needed first is **an agreed definition of what is being counted**.

01-1. What the provided data actually looks like

Table 1. The organizers' four master datasets — measured after loading (reference date 2026-08-21; foreign ETFs 2026-08-22)

Master / view	Rows	Actual products	Why rows and products diverge
Domestic bonds `v_bond_kr`	21,882	20,497 issues	One `pd_no` comes with both exchange-traded and OTC rows, and OTC holdings appear as multiple rows **per LOT** (`info_seq`)
Domestic ETFs `v_etf_kr`	1,780	ETF 1,235 / ETN 545	**Two kinds, ETFs and ETNs,** share the same columns in one table
Foreign ETFs `v_etf_gl`	6,037	ETF 5,972 / ETN 65	Same (foreign ETFs + foreign ETNs)
Public funds `v_fund_pub`	23,676	17,795 funds	1 row = 1 **fund share class**. Classes A/C/S each take a row of their own
Total	**53,375**	**—**	**+ 22,599 domestic-ETF constituent rows (externally collected, reference date 2026-09-05)**

Figure 1. Why this is not an SQL problem — four structural difficulties

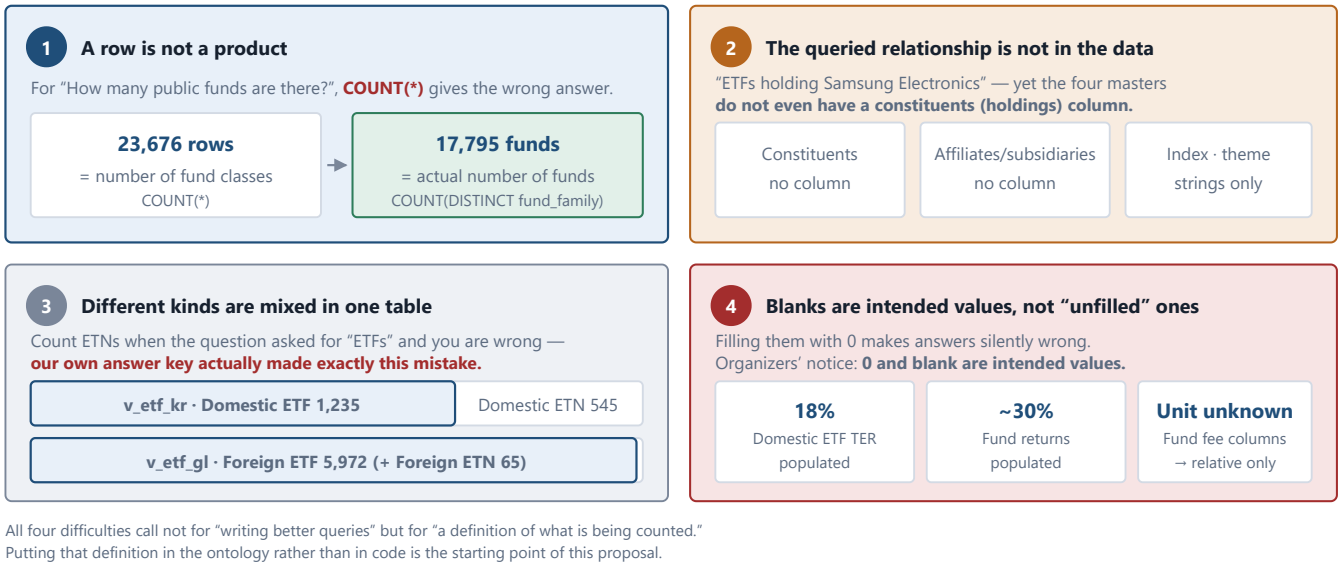


Figure 1. Difficulties ① and ③ are problems of definition, ② is a problem of missing relationships, and ④ is a problem of silent errors. ② is impossible from the very first step with SQL alone, so a knowledge graph is required; ①, ③ and ④ only become repeatable once "what is being counted" is made explicit in the ontology.

01-2. The hardest difficulty — there is no way to check the right answer

The organizers do not disclose the evaluation questions or their answers (task description: no reference question set included). So **we have to build the answer key ourselves**. And therein lies a trap. If the answer key is computed **with the same code** as the system, then whenever the code is wrong, the answer and the key are wrong **together** — and grading shows green.

That actually happened. The knowledge graph **had inflated the number of constituent relationships twofold**, and it was still the "right answer" — because the answer key had been computed with the same code as the graph. After this incident,

we made it standard procedure to **recount the numbers produced by SQL through the graph, and the numbers produced by the graph through SQL, and compare them** (02-3, 05-1).

Table 2. Six incidents we actually ran into — every one of them passed with no exception, no log and no red test

Incident	Symptom	Cause	Fix (what remains in the design)
Relationship count inflated 2×	The number of constituent relationships was doubled, yet grading **passed**	The answer key was computed with **the same code** as the graph	**SQL ↔ graph cross-validation made a procedure.** Reconciled on 5 companies: Samsung Electronics 229/229 · EcoPro BM 42/42 · LG 26/26 · KT 48/48 · Samsung Electronics (pref.) 19/19
677 constituent rows misattributed	LG mapped to Shinhan Financial Group, KT to SKT	**The conversational name matcher was reused for bulk loading.** `"lg"` matched as a substring of `"shinhanfinancialgroup"`	Companies are identified only by the **stock code** supplied by the source (`company_code` , 79% populated). Name-similarity matching removed (`tests/test_constituent_keys.py`)
Misuse of `rdfs:domain`	ETN rows misclassified into the ETF class	`domain` **is an inference rule, not a constraint.** Because it asserts "whoever uses this property is of this class," it fabricates wrong facts instead of filtering them out	Where a constraint is needed: **SHACL NodeShapes** (13). Domains fixed by adding `fp:DomesticListedProduct` / `fp:ForeignListedProduct` so that every row type in the table is covered
A capped count reported as the total	Answered "200 in total," while the actual number was 15,837	The display-row cap (LIMIT 200) leaked out as the total	The evidence block reports the two separately as **"N in total (200 shown)"** (`agent/answer.py`)
Answers without tool calls	The model **invented products that do not exist**. Of 5 runs of the same question, the runs that called tools were **all correct**; the runs that did not were **all wrong**	When the model judges that it can answer from pre-training knowledge, it skips the lookup	**An answer submitted with zero lookups is rejected** and sent back. In the 54-question run this guard **fired 5 times** (`agent/orchestrator.py`)
Entity resolution not applied to SQL	**133 ETFs holding Naver were answered as "no data"**	Entity resolution was wired only into the graph path; the SQL path queried the user's own spelling (`'네이버'` , Naver written in Hangul) verbatim	Fixed so that pre-analysis also hands **the store's actual codes and spellings** to the SQL path (`agent/routing.py`)

"After you change something, count the results again."

None of the six incidents above threw an exception, left a log entry or turned a test red. They are not caught by looking at success or failure; they are caught only by **counting again**. So for every load and fill step we made it procedure to **recompute and compare the record counts before and after the operation**.

02 Proposed Approach

> This chapter answers, in order, the three questions set by the organizers — ① **what specialized data did we collect and cleanse, and how**, ② **what ontology engineering did we apply**, and ③ **which retrieval methodology did we choose, and why did we design it that way**.

02-1. What specialized data did we collect and cleanse, and how

Principle — external collection only for "items with no column at all in the provided data"

The rule is **compute it from the provided data**. External collection is not an exception to that rule: it was applied to only one item that cannot be computed because the provided data **has no column for it at all** (ETF constituents). Whether the rule was actually kept can be checked from the provenance breakdown of the knowledge graph.

Figure 2. Where do the knowledge graph's 1,082,874 triples come from — measured provenance

965,823 asserted + 117,051 inferred · 73 predicates



- **Organizers' masters** — directly from the 4 masters
- **Ontology · inference** — TBox merge + rule materialization
- **Our dictionaries** — 77 managers · 48 themes · 97 firms
- **External collection** — domestic ETF holdings (sole)

Triples rooted in the organizers' data: **85.9%** (58.9 + 27.0). Only **6.0%** came from outside — and only for the one item with no column in the provided data.

Figure 2. The provenance shares were measured when the graph held 1,074,824 triples; affiliate derivation later raised the total to 1,082,874 (the shares were not re-measured, so we state that they are as of the time of measurement).

Cleansing — three lessons that came out of failures

Table 3. Three cleansing cases (before → after, all measured)

Target	How we found it	How we fixed it	Result
Definition of purchasable `is_sellable`	An organizers' notice (2026-08-24) **invalidated** the values of `BUYABLE_QUANTITY`	Redefined as "not yet redeemed = purchasable." ETL, tests and prompts were changed together	Bonds 634 → **21,866** an item on which evaluation answers change substantially
Fund manager `manager_name`	A full audit of per-view NULL ratios found **25.4% (6,016 rows) missing**. The cause: we had been **guessing the manager from the fund-name prefix**	Derived from `or_co_xtn_itt_cd` (manager institution code), which the organizers supply **100% populated**. The majority spelling for each code is used as the canonical name	17,660 rows (74.6%) → **21,592 rows (91.2%)** provenance recorded in `manager_source` name 17,660 / code 3,932 / NULL 2,084
Constituent identifier `company_code`	While reconciling SQL and graph constituent counts, we found **677 misattributed rows** (LG → Shinhan Financial Group, KT → SKT)	**Removed** the conversational alias matcher from bulk loading; only the stock codes supplied by the source (`itemCode`) are used	**79%** of rows carry a stock code 0 name-similarity matches

> **How to catch silent errors**
>
> One fill step in the pipeline was **reporting success while filling no rows at all**: when `pandas`' `Series.map` returned `None`, it became `NaN`, and `NaN` is truthy. There was no exception, no log entry, and the tests were green. So every fill step now looks **not at success or failure, but at the number of rows filled before and after the operation**.

What we chose not to fill — if the evidence is not in the provided data, we do not fill it in

- **A manager-rename mapping table**: old and new brands are observed together under one institution code, but the provided data **has no fund inception date**, so there is no basis for deciding which is the current name. Pulling in an external table would break the rule, so we use the majority spelling as is — even when it is the old brand.

- **Domestic ETF TER (18% populated) · fund returns (about 30% populated) · fund fee units**: these are values the source did not provide. We do not estimate them and answer "no data"; because the unit of the fund fee columns is undisclosed, we allow **only relative comparisons between classes**.
- **Return outliers**: some 1-year returns exceed 200%, but **since these are values supplied by the organizers, we use them as is, unmodified**.

External collection — constituents, and the affiliates derived from them

We collected the top constituents of **1,162 of the 1,235 domestic ETFs (22,599 rows)**, **loaded them into the SQLite table** `etf_constituent`, and generated the knowledge graph's `hasConstituent` **from that table**. Because both paths read the same source, SQL results and graph results do not diverge.

From there we **derived affiliate relationships**. The constituents of group-stock ETFs (`TIGER 삼성그룹` [Samsung Group], `PLUS 한화그룹주` [Hanwha Group stocks], etc.) are precisely the member companies of that business group, so we materialized them as `affiliateOf` — **608 edges across 65 companies in 8 groups, with 0 false positives** (`derive_group_affiliates` in `kb/graph.py`). **This, however, is the fact of belonging to "the same business group," not "equity ownership."** The data has no shareholding ratios, so the levels cannot be told apart, and answers therefore also say **"affiliate."** The hand-written `subsidiaryOf` consists of **just 4 edges**, and we do not claim "multi-level subsidiary traversal" as a general capability.

02-2. What ontology engineering did we apply

> Six TTL files are read at startup to automatically generate ① the **schema card** shown to the LLM, ② the **validator** that filters the values in a question, and ③ the **inference rules** materialized into the graph. As a result, the cost of adding a new product type drops to **one view + one TTL**.

Figure 3. The ontology — class hierarchy (left) and runtime binding (right)

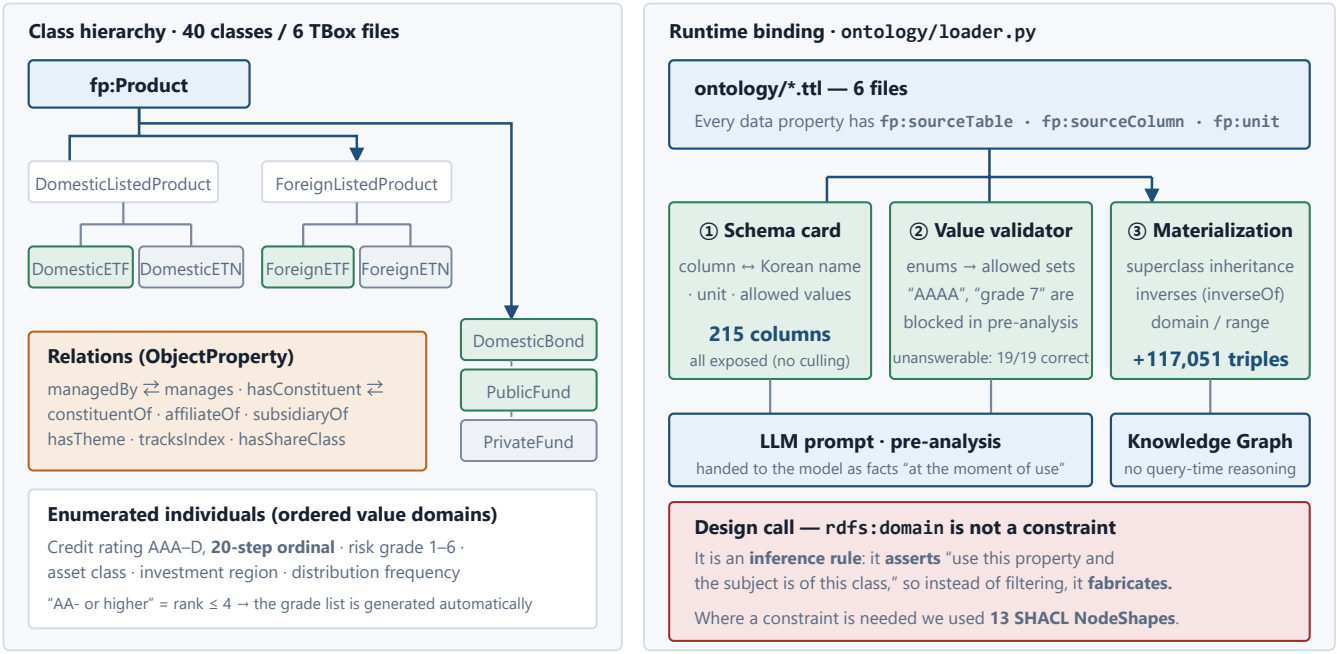


Figure 3. The left side is the class hierarchy; the right side is what those definitions become at runtime. Of the **261 property definitions, 215** are linked to actual view columns (`fp:sourceTable` / `fp:sourceColumn`). **Unlinked properties do not appear on the schema card, and a column missing from the schema card is, for the model, as good as nonexistent** (measured evidence in 02-3).

Table 4. The six ontology files (submitted as `ontology/*.ttl`)

File	SHACL	Contents
`common.ttl`	4	Top class `fp:Product`; listed-product split `DomesticListedProduct` / `ForeignListedProduct`; asset manager · issuer · company · theme · index · fund family; enumerations for risk grade (1–6) · asset class · investment region · distribution frequency; common relations (`managedBy` · `hasConstituent` · `affiliateOf` · `subsidiaryOf` · `hasTheme` · `tracksIndex`)
`bond_kr.ttl`	2	`fp:DomesticBond` — credit rating scheme (AAA–D, **20-step ordinal**); enumerations for bond type · rate type · interest payment; yield and duration properties
`etf_kr.ttl`	2	`fp:DomesticETF` / `fp:DomesticETN` — strategy (physical · synthetic · active), leverage multiple, period returns · volatility, distribution information
`etf_gl.ttl`	2	`fp:ForeignETF` / `fp:ForeignETN` — total expense ratio, AUM, exchange, replication method, inverse · leverage
`fund_pub.ttl`	3	`fp:PublicFund` / `fp:PrivateFund` / `fp:ShareClass` — management attributes, benchmark, class fees · sales channels, asset-allocation ratios, manager institution code and fill provenance (`manager_source`)
`etf_constituent.ttl`	0	Domestic ETF constituents — holding relationship, stock code · weight · cash-equivalent flag · reference date. The graph's `hasConstituent` is generated from this view
Total	**13**	**40 classes · 261 property definitions (of which 215 are linked to actual view columns)**

A limitation we want to state — the 13 SHACL NodeShapes **are declared, but runtime validation is not applied** (`pyshacl` is not among the dependencies). For now, SHACL serves as a specification of the constraints; the actual blocking of values is done in the pre-analysis stage by the validator generated from the ontology's enumerations. Adopting `pyshacl` is left as an extension item (05-3).

02-3. Which retrieval methodology did we choose, and why did we design it that way

Five paths — but which ones to switch on is decided deterministically

Different elements of a query take different paths — numeric filters, sorting and aggregation go to SQL; relationship traversal to the graph; name and description matching to BM25; background knowledge to the documents. **We did not leave this choice to the model**. Before the LLM is called, pre-analysis draws up a **retrieval execution plan**, runs in advance only the stores the question needs, and **injects the resulting rows into the first prompt as facts** (`plan_retrieval` in `agent/routing.py`).

Figure 4. Five federated retrieval paths — which question goes where

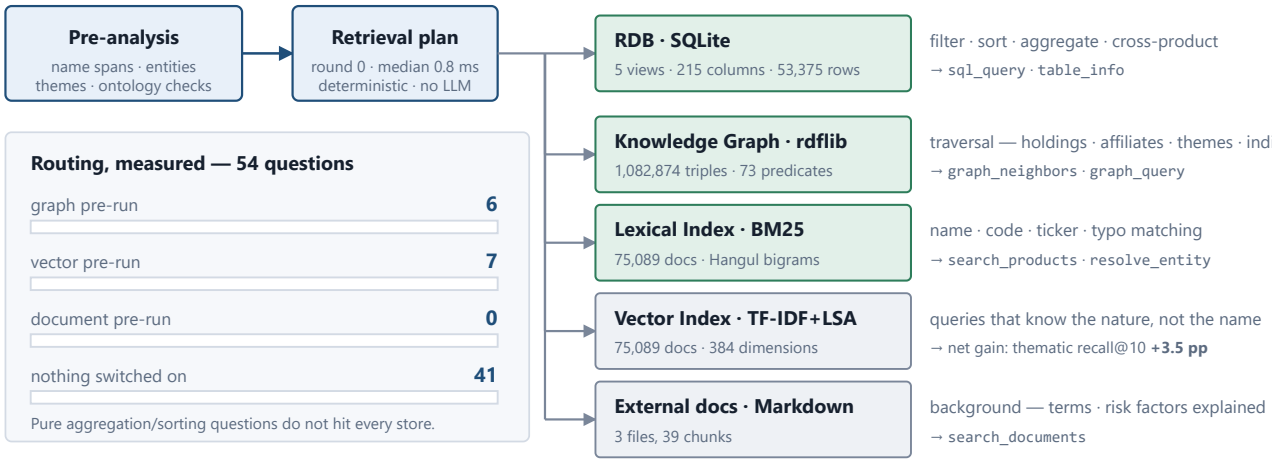


Figure 4. Federated retrieval, yet it does not always hit all five stores. Median planning time is **0.8 ms**, and which stores were chosen, and why, is recorded verbatim in `think_trace`.

Hybrid fusion — why we dropped RRF, and how large the alternative's gain measured

At first we fused BM25 and vector results on an equal footing with **RRF (Reciprocal Rank Fusion)**. RRF lifts documents on which both retrievers agree, but in this corpus we saw cases where **unrelated funds that each retriever had placed mid-list**

pushed out the top lexical match ("products related to battery cathode materials" → rank 1 was an unrelated fund of funds). If the top candidate is wrong, answer generation is contaminated, so the final structure is: **lexical results take the head of the list, and the bottom ~30% of the slots are reserved for documents "found only by semantic search."**

Figure 5. Fusion structure (left) and the size of its gain (right) — measured on 1,313 queries

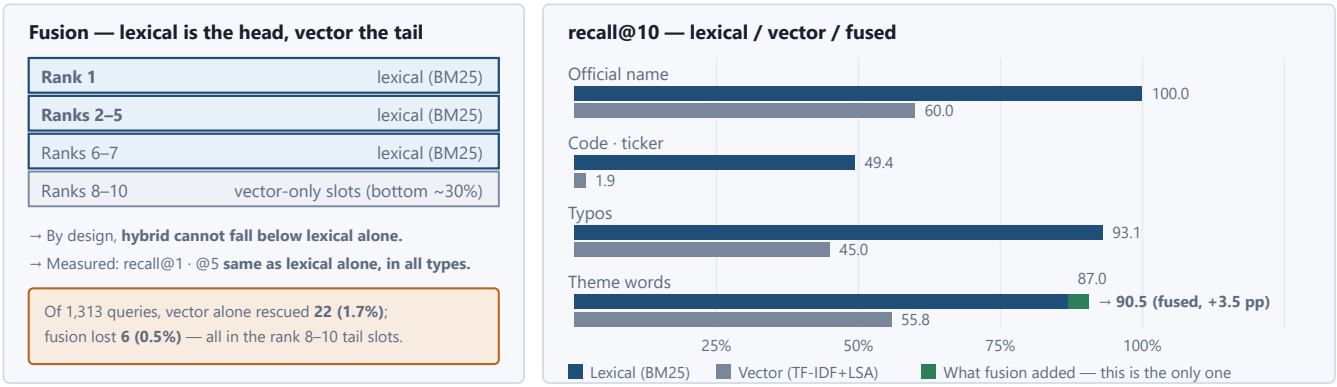


Figure 5. We did not write that "semantic search absorbs typos and synonyms." **Typo tolerance is provided by BM25's Hangul bigrams** (lexical 93.1% vs. vector 45.0%). TF-IDF+LSA is based on lexical co-occurrence, not semantic embeddings, so this is about the ceiling for this method. Reproduce with: `python scripts/audit_vector.py --sections BC`.

The model does not ask about the schema — so we changed the prompt design

In the 54-question run, the tools the model actually called were `sql_query` **49 times**, `graph_neighbors` **7 times** and `search_products` **twice**; `search_documents`, `table_info` and `graph_query` were called **0 times each**. The model does not ask about the schema. Therefore **a column that is not in the prompt from the start effectively does not exist for the model**. In fact, `has_buy_yield` was in the column list but had no ontology definition, so it did not appear on the schema card — and the model never used that column. This is why we decided **to expose all 215 provided columns rather than a curated selection**, and the measurement showing that latency does not grow even when the prompt grows 22-fold (05-2) backs that decision.

"What was handed over as fact at the moment of use was followed (5/5); prompt rules given in advance were ignored (4/4)."

Repeated measurements gave the same result. Adding more warnings had no effect; **handing the model the schema card, the pre-retrieved results and the lookup keys at the very moment it uses them** did.

The clearest case: in a cross-product query, the warning "foreign ETFs have no 1-year return, so exclude them" was ignored, but once **we removed foreign ETFs from the `UNION` in the example SQL**, that failure disappeared — the warning was not read, while the example was copied. So instead of adding rules, we provide **examples and facts in which the rules are already reflected**.

Answer generation — answers without a lookup are refused

When the model answers without calling tools, it **makes up products that do not exist** (Table 2). So `final_answer` enforces three states — `answered` / `cannot_confirm` / `clarify` — through the tool schema, and **the server rejects and sends back any submission made without a single lookup**. In the 54-question run this guard **fired 5 times**.

03 System Architecture

> Load → index → retrieve → generate. Because the four stages reference **one and the same ontology**, a definition made during loading becomes document text in the index, then the schema card for retrieval, and then the value validator for generation.

Figure 6. The full pipeline — load → index → retrieve → generate

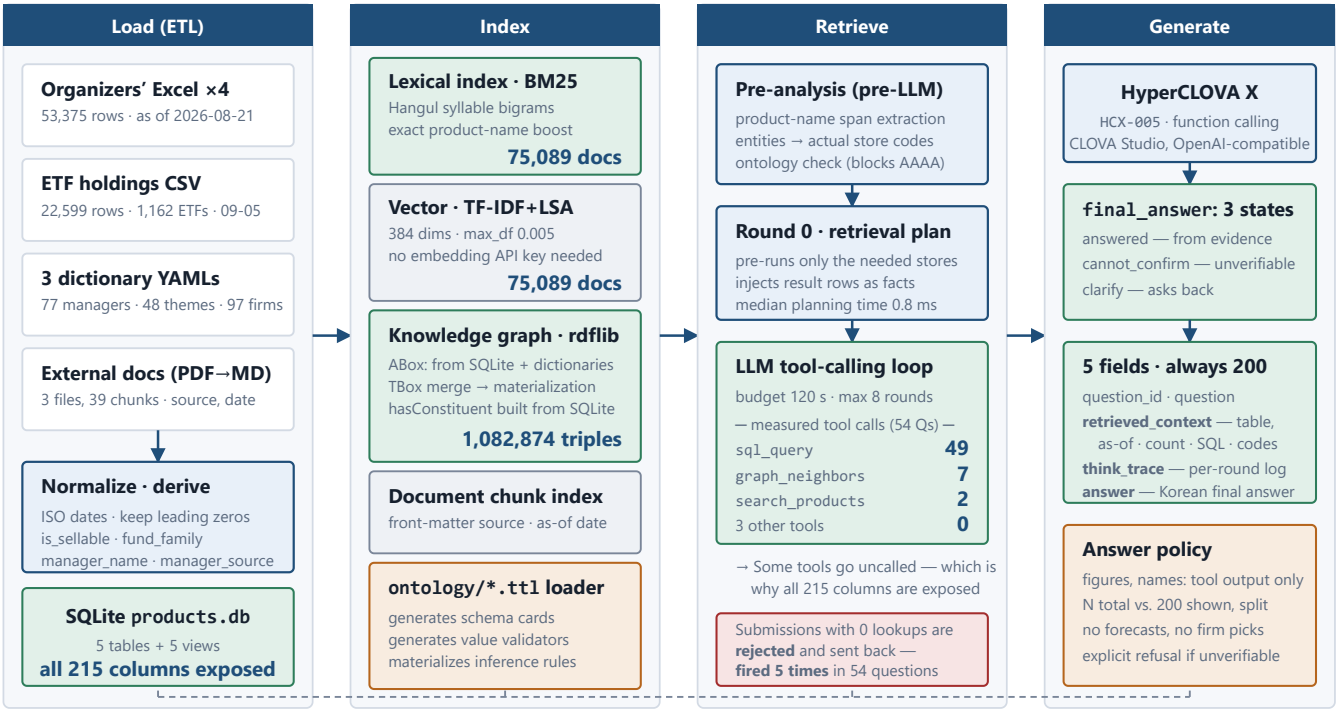


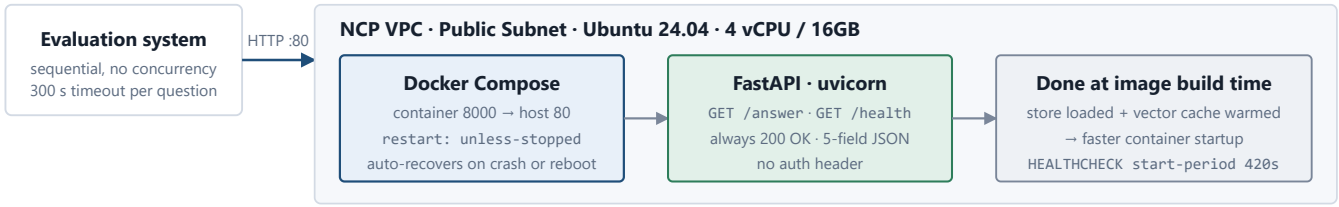
Figure 6. The orange dotted line means the ontology runs through all four stages — the same TTL becomes the derived definitions in loading, the inference rules in indexing, the schema card in retrieval, and the value validator in generation.

Table 5. Components · roles · code locations

Component	Role	Code
ETL	Excel masters + constituents CSV → SQLite. ISO date conversion, on-sale flag, fund-family key, asset-manager normalization (derived from institution codes), 5 views · all 215 columns exposed	`src/finagent/etl/`
Ontology	Loads the TBox at runtime and generates the schema card (column ↔ Korean meaning · unit · allowed values) and the validators (value domains for credit rating and risk grade)	`ontology/loader.py` `Ontology.schema_card()` · `resolve_measure()`
Knowledge Graph	Builds the ABox from SQLite + dictionaries, merges the TBox, then materializes inferences. Constituents are generated from the SQLite table, so they cannot diverge from SQL. Includes affiliate derivation	`kb/graph.py` `KG.build()` · `_materialize()` · `derive_group_affiliates()`
Lexical Index	BM25 + Hangul syllable bigrams, exact product-name boost. Entity resolver (exact match · fuzzy candidates · company relations)	`kb/search.py` · `entity.py` · `lexicon.py`
Vector Index	TF-IDF (words + Hangul bigrams) → Truncated SVD to 384 dimensions → cosine. Deterministic, with no embedding key. Switchable via `VECTOR_BACKEND=openai`	`kb/vector.py`
Retrieval Routing	The retrieval execution plan of round 0. Deterministically pre-runs only the stores needed and injects the results as facts	`agent/routing.py` `plan_retrieval()` · `signals_for()`
Agent	Tool-calling LLM loop, time budget, evidence and trace collection, answer policy. Rejects answers submitted with zero lookups	`agent/orchestrator.py` · `tools.py` · `answer.py`
API	FastAPI `GET /answer`, `GET /health`. Always 200 + 5 fields	`src/finagent/api.py`

Deployment — tailored to how the organizers call the evaluation

Figure 7. Deployment — Naver Cloud Platform



The organizers' notice specifies plain HTTP on the standard port, so we put no certificate or reverse proxy in front (a Caddy + Let's Encrypt setup is also possible with `--profile https`, but it requires a domain).

Figure 7. `.env` is listed in `.dockerignore`, so it is not baked into the image (protecting the API key); it is read at runtime via `env_file`.

04 Key Functional Flow

> Data collection → ontology design → graph construction → retrieval. We show the four stages as **the path that a single real question takes through them**.

Figure 8. The four stages, their outputs, and their verification points

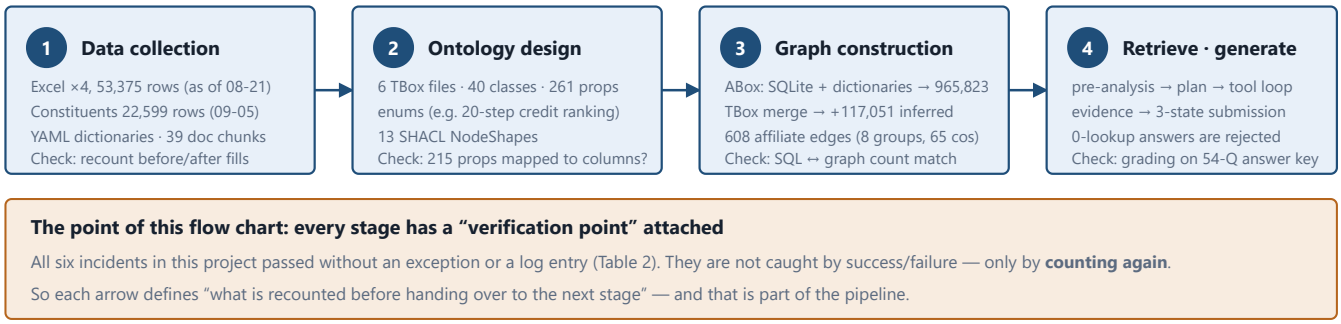


Figure 8. The four stages run in sequence, but the stage-3 graph takes both the stage-2 ontology and the stage-1 SQLite store as its sources. Constituent relationships are generated **from the SQLite table**, not from the graph.

How a single question is processed — a real trace

Below is the actual execution record of question `G-015`, difficulty **[Hard]** (`data/eval/truth_results.jsonl`). All numbers and SQL in the figure are **copied verbatim from the actual response**; the question and the Korean answer text are shown in English translation.

Figure 9. "Of the domestic ETFs holding Samsung Electronics, name the 3 with the largest net assets" — 15.6 s, correct

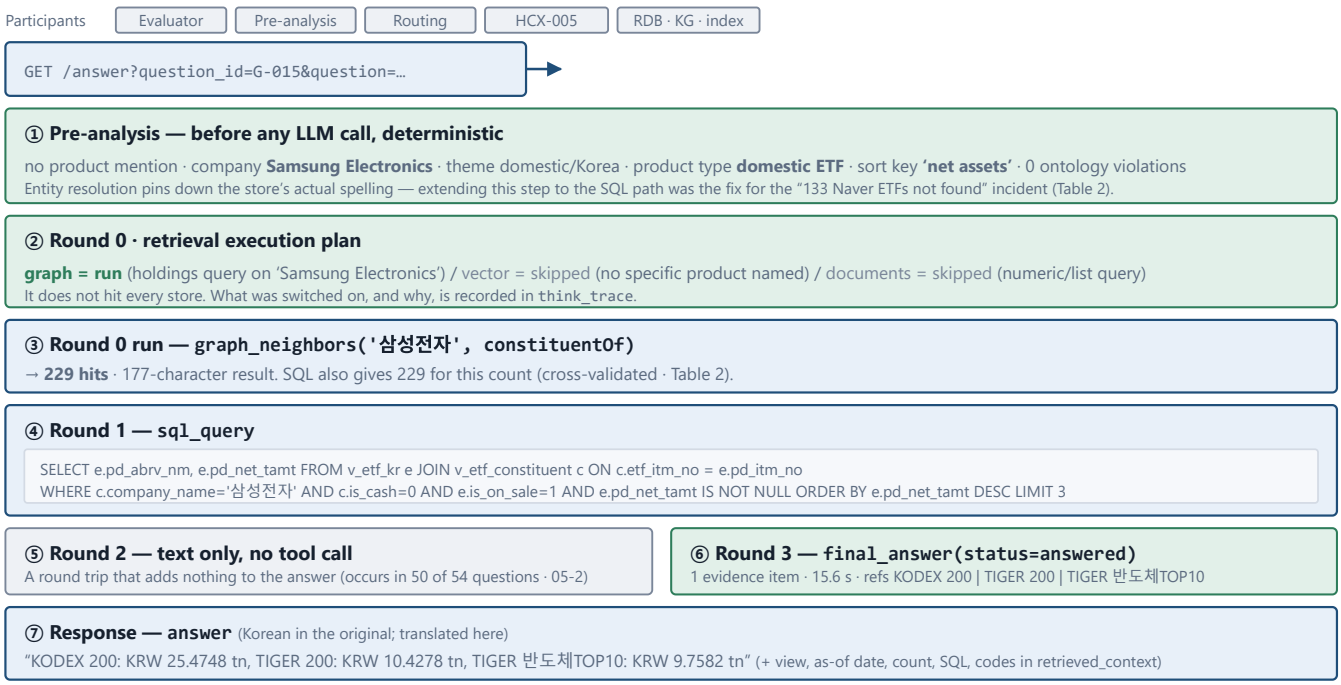


Figure 9. This question is **impossible from the very first step with SQL alone** — the four masters have no constituents column (Figure 1, difficulty ②). The graph resolves the relationship, SQL finishes the sorting and aggregation, and **because both paths draw on the same SQLite table, the numbers do not diverge**. (`'삼성전자'` is the stored company name of Samsung Electronics; `TIGER 반도체TOP10` is TIGER Semiconductor TOP10.)

Table 6. The answer-policy state machine — mapped to the organizers' evaluation rules

State	When it is chosen	What the server guarantees
`answered`	When the values are confirmed by tool results	Figures and product names come **only from tool results**. The evidence (Korean view name · reference date · count · SQL/SPARQL · product codes) must be recorded in `retrieved_context`. When capped, reported separately as **"N in total (200 shown)"**
`cannot_confirm`	A nonexistent product, company or value; outside the data's range; or the product type has no such data	Starts with "This cannot be confirmed from the provided data." Violations of the ontology's allowed values (credit rating AAAA, risk grade 7) are **blocked in pre-analysis**. **Measured: 19/19 correct**
`clarify`	When the conditions are too ambiguous to pin down an answer	Responds with a clarifying question (ending in a question mark)
Common guards	**Submissions with zero lookups are rejected and sent back (fired 5 times in 54 questions) · no return forecasts or definitive recommendations · when the budget (120 s) is about to run out, the loop is force-stopped and answers with the evidence gathered · even on an internal error, 200 + the same schema instead of a 500**	

05 Expected Impact · Scalability

05-1. Measured performance

> **About the answer key —** the organizers did not disclose the evaluation questions or their answers (task description: no reference question set included). The figures below are therefore based on **54 self-built questions whose answers are computed from the provided data via SQL and the graph**, not on performance against the organizers' evaluation set. The answer key is recomputed from the data at grading time, so when the data is updated the answers are updated along with it.

Table 7. Grading results on our own 54-question answer key — measured with `scripts/eval_truth.py`

Difficulty	Correct / Qs	Accuracy	Product type	Correct / Qs	Response time	Value
Easy	8 / 10	80.0%	**Domestic ETF**	**14 / 14**	Mean	**20.6 s**
Medium	14 / 16	87.5%	**Bonds**	**9 / 9**	Median	14.2 s
Hard	6 / 9	66.7%	**Funds**	10 / 12	Max	87.4 s
Unanswerable	**19 / 19**	**100%**	**Foreign ETF**	5 / 7	Over 60 s	1
Total	**47 / 54**	**87.0%**	**Cross-product**	**8 / 11**	**Over 300 s**	**0**

All 19 unanswerable questions were answered correctly — because the ontology value validator blocks nonexistent values in pre-analysis (02-2, Figure 3 ②). The weakest areas are **cross-product queries (8/11)** and **Hard difficulty (6/9)**.

Table 8. All 7 wrong answers

Q	Difficulty	Question (translated)	What went wrong · diagnosis
G-012	Medium	How many currency-hedged overseas-bond public funds are currently on sale?	Expected value 63 missing from the answer — ambiguous definition: **"fund class or fund?"** (06-D)
G-027	Easy	Among public funds on sale, how many have risk grade 1?	Expected value 444 missing — same cause
G-043	Hard	Across domestic ETFs and public funds on sale combined, how many products have risk grade 1?	Expected value 667 missing — same cause + cross-product query
G-044	Hard	How many domestic ETFs invest in bonds, and how many bond-type public funds are there, respectively?	0/2 — in the cross-product query, the two product types use different criteria for "bond-type"
G-032	Medium	Among foreign ETFs, considering only bond products, which 3 have the largest AUM?	0/3 — interpretation of the foreign-ETF asset-class column
G-037	Hard	How many domestic ETFs hold SK hynix?	Expected value 214 missing — a scope limit: the constituents **cover only the top holdings**
G-034	Easy	What is the name of the asset manager of Invesco QQQ Trust, Series 1?	Mismatch in the manager's official name (`Invesco Capital Management L…`) — the edge of exact-string-match grading

4 of the 7 share the same cause (an ambiguous definition of what to count). This is not a failure of the model but **a question the ontology has not yet answered** — whether "the number of funds" counts classes or families is a matter for **definition**, not for the data, and making that definition explicit in the ontology is the next piece of work (06-D).

05-2. The structure of latency — what consumes the time

Figure 10. Prompt size does not create latency (left) · round trips do (right)

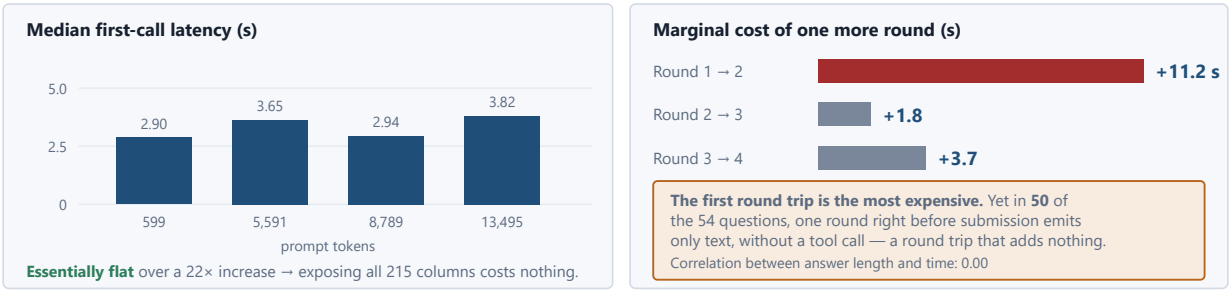


Figure 10. The 35-second calls observed earlier were not caused by prompt size but by **rate-limit waits on the test-app tier**. These two measurements led to two different conclusions: "do not curate the columns — expose them all" and "reduce the number of round trips."

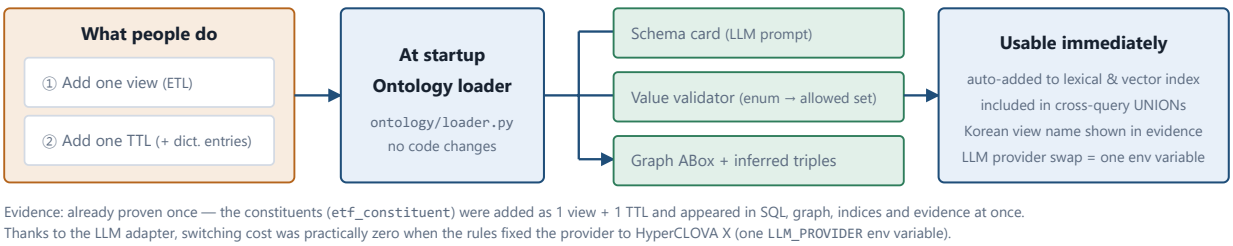
05-3. Points of business application

Table 9. Business application — what is already built into the architecture

Application	What this system already does	Remaining work
Advisory support	Product search, comparison and condition filtering in natural language. Every answer carries **the referenced view's Korean name · reference date · count · product codes**, so an advisor can check the source immediately	A connector to the in-house product master; integration with customers' holdings
Compliance	**Ungrounded answers are structurally impossible** (zero-lookup submissions are rejected) · no return forecasts or definitive recommendations · explicit refusal when unverifiable. `think_trace` provides an audit trail as is	Move the prohibited terms and mandatory wording of internal review rules into ontology constraints
Product data quality	SQL ↔ graph cross-validation **detects loading errors automatically**. Missing-value fills record their provenance in columns such as `manager_source`	Adopt `pyshacl` to promote the 13 SHACL NodeShapes to **actual validation at load time**
Internal knowledge search	The ontology holds "what each column means" in Korean — it doubles as onboarding documentation for new staff	Expand the unstructured documents, such as terms and conditions and prospectuses (currently 3 files, 39 chunks)

05-4. Scalability — the cost of adding a new product type

Figure 11. Adding a new product type — people do two things, the loader builds the rest



Evidence: already proven once — the constituents (`etf_constituent`) were added as 1 view + 1 TTL and appeared in SQL, graph, indices and evidence at once.
Thanks to the LLM adapter, switching cost was practically zero when the rules fixed the provider to HyperCLOVA X (one `LLM_PROVIDER` env variable).

Figure 11. Adding a TTL grows the prompt, the validator and the graph together; none of the three needs separate attention.

Directions for extension

- **More product types** — overseas bonds · wrap accounts · trusts · derivative-linked securities (ELS/DLS) each attach as one view + one TTL. The procedure has already been validated on the four masters.
- **More relationships** — `affiliateOf` currently consists of **608 edges** derived from group-stock ETFs. If shareholding data becomes available, the same pipeline can formally derive `subsidiaryOf`, promoting "affiliate" to "subsidiary."
- **Better retrieval** — switch to commercial embeddings with `VECTOR_BACKEND=openai` (no code change needed). This leaves room to exceed the current TF-IDF+LSA ceiling, but **we do not claim an improvement we have not measured**.
- **Stronger validation** — adopt `pyshacl` so the 13 SHACL NodeShapes, so far only declared, are validated at load time.
- **Sharper definitions** — declare **"what counts as one"**, the cause of 4 of the 7 wrong answers, as explicit cardinality in the ontology (05-1, Table 8).

06 Appendix

06-A. Ontology schema summary

Table 10. Ontology composition summary (`ontology/*.ttl` · 6 files)

Item	Count	Notes
TBox files	6	`common` · `bond_kr` · `etf_kr` · `etf_gl` · `fund_pub` · `etf_constituent`
Classes	40	Top-level `fp:Product` → listing type → product type. `DomesticListedProduct` / `ForeignListedProduct` added to cover ETNs
Property definitions	261	Of these, **215** are linked to actual view columns via `fp:sourceTable` / `fp:sourceColumn`
Enumerated individuals	—	Credit rating AAA–D (**20-step ordinal**) · risk grade 1–6 · asset class · investment region · distribution frequency. Expressions such as "AA- or higher" are expanded automatically by ordinal comparison
Object properties	—	`managedBy` ⇄ `manages` · `hasConstituent` ⇄ `constituentOf` · `affiliateOf` · `subsidiaryOf` · `hasTheme` · `tracksIndex` · `hasShareClass` (reverse directions generated automatically from the `owl:inverseOf` declarations)
SHACL NodeShape	13	**Declared only; not validated at runtime** (no `pyshacl` dependency) — see 06-D
ABox (built at startup)	965,823	Generated from SQLite + dictionaries (products · managers · indices · themes · companies · fund families · holdings)
Materialized inferences	117,051	Superclass inheritance · inverse relations · domain/range type inference. **Zero inference cost at query time**
Knowledge graph total	**1,082,874**	**73 predicates (previously 5,082 → cleaned up by folding holding weights into a column instead of per-company predicates)**

06-B. API specification

Table 11. Endpoint · request · response

Item	Value
Endpoint	`GET http://223.130.161.217/answer` — path fixed at `/answer`, no auth header, no POST body
Request parameters	`question_id` (string, required · echoed back as is) · `question` (string, required · URL-encoded). Undefined parameters are ignored
Response	`200 OK` · `application/json; charset=utf-8` · **all 5 fields are strings** `question_id` · `question` · `retrieved_context` · `think_trace` · `answer`
`retrieved_context`	Each `[근거 n]` ("evidence n") block carries the referenced view (Korean name · view name) · data reference date · **count** · the executed SQL/SPARQL/search terms · the referenced product codes and names. When capped, it is marked separately as **"N in total (200 shown)"**
`think_trace`	Numbered processing steps: pre-analysis (entities · themes · ontology validation) → round-0 routing decision and its reasons → each round's tool calls and result sizes → final status · elapsed time
Error handling	**HTTP 500 is never returned.** Even on internal errors, LLM failures or timeouts, the response is 200 + the same schema, carrying the evidence gathered so far
Auxiliary endpoint	`GET /health` — status · table reference dates · index sizes · graph `kg_asserted` / `kg_inferred` · vector backend
Response time	Per the organizers, more than 300 s counts as no response. The server's internal budget is **120 s · max 8 rounds**; when it is exceeded, the server answers immediately with the evidence gathered. **Measured: 0 cases over 300 s**

Example call

```
curl -G "http://223.130.161.217/answer" \
  --data-urlencode "question_id=Q-001" \
  --data-urlencode "question=현재 판매가능한 원화채권 중 AA- 이상 종목 알려줘"
# question = “List the KRW bonds currently available for purchase that are rated AA- or higher”
```

Response (same schema · 200 OK even for unverifiable queries; the Korean string values are shown in English translation)

```
{
  "question_id": "Q-031",
  "question": "Find bonds with a credit rating of AAAA",
  "retrieved_context": "No data referenced",
  "think_trace": "1. Pre-analysis: no product mention; 1 ontology violation\n2. Round 1:
final_answer(status=cannot_confirm)\n3. Final: status=cannot_confirm, 0 evidence items, elapsed 6.1s",
  "answer": "This cannot be confirmed from the provided data. Credit rating 'AAAA' is not a valid value.
Valid credit ratings: AAA, AA+, AA0, AA-, A+, … D"
}
```

06-C. Evaluation reproduction procedure

Table 12. Steps the graders can run as is

Purpose	Command	Notes
Environment setup	`pip install -r requirements.txt` `cp .env.example .env`	Python 3.11+ · 4 GB of memory or more
Data loading	`python scripts/build_store.py`	Excel → SQLite, about 1–2 min
(Optional) warm-up	`python scripts/warmup.py`	Vector cache. Cuts later startups by about 80 s
Run the server	`uvicorn finagent.api:app --host 0.0.0.0 --port 8000 --app-dir src`	Builds the indices, vectors and graph + inferences at startup (about 1 min 30 s with the cache)
Docker (same as the evaluation server)	`docker compose up -d --build`	Loading + warm-up are done at image build time. Container 8000 → host 80
Answer-key grading	`python scripts/eval_truth.py --compute` `python scripts/eval_truth.py --base http://127.0.0.1:8000`	The first line recomputes only the answer key from the data, **without the LLM**. Results go to `data/eval/truth_results.jsonl`
Retrieval audit (no LLM needed)	`python scripts/audit_vector.py --sections BC`	Re-measures lexical/vector/fused recall on 1,313 queries. **Fixed seed**
Inspect the stores directly	`python scripts/explore.py health` `python scripts/explore.py pre "신용등급 AAAA인 채권 찾아줘"`	Check each store and the pre-analysis output directly, without an LLM key (the sample question means “Find bonds with a credit rating of AAAA”)
Tests	`python -m pytest -q`	**194 passing** (stub LLM · small store). `-m slow` for full ETL verification

06-D. Data and system limitations

> This is the list of **what this system cannot do**. Each item below is actually used as the grounds for a `cannot_confirm` response.

Table 13. Complete list of limitations

Item	Details
Basis of the accuracy figures	Because the organizers did not disclose the evaluation set, the figures are based on **54 self-built questions**. They are not performance on the organizers' evaluation set.
Ambiguous definition of "how many"	**One fund row = one class**, and **a bond has several LOT rows per issue**. Whether "how many funds" means the number of classes or of families is decided by definition, not by the data. **4 of the 7 wrong answers stem from this** (Table 8).
Net contribution of the vector leg	Limited to **+3.5 pp recall@10 on thematic queries** (87.0 → 90.5). recall@1 · @5 are identical to lexical alone, and **typo tolerance comes from the BM25 Hangul bigrams (93.1%), not from the vector index (45.0%)**. It exists and works as a required component, but in this corpus the range in which semantic search beats lexical search is narrow.
Affiliate ≠ subsidiary	The 608 `affiliateOf` edges derived from group-stock ETFs (8 groups, 65 companies, 0 false positives) **record membership of the same business group, not equity ownership.** The data has no shareholding ratios, so the levels cannot be told apart, and answers therefore also say "affiliate." The hand-written `subsidiaryOf` consists of **just 4 edges**, and we do not claim "multi-level subsidiary traversal" as a general capability. Unlisted affiliates are invisible, and only 2 Hyundai Motor Group companies are captured (the matching tokens were kept narrow to avoid false positives). SK Group is not covered because its ETF has no constituent rows.
SHACL not validated at runtime	The 13 NodeShapes **are declared, but because `pyshacl` is not a dependency, no actual validation is performed at load time.** Value blocking is handled in the pre-analysis stage by the validator generated from the ontology's enumerations.
Constituent coverage	Constituents exist **only for domestic ETFs** (collected for 1,162 of 1,235). There are no constituent data for foreign ETFs or public funds, and such questions are answered as unverifiable. Also, **only the top holdings are included**, so this is not a complete holdings list (the cause of wrong answer G-037).
Reference-date mismatch	The constituents' `as_of` is **2026-09-05**, while the product masters are as of **2026-08-21** (foreign ETFs 08-22). When an answer mixes the two sources, the difference in timing is stated as well.
Low-coverage columns	Only **18%** of domestic ETF TERs and **about 30%** of fund returns are populated. The unit of the fund fee columns is **undisclosed**, so only **relative comparisons** between classes are allowed. Foreign ETFs have no 1-year return, so they are excluded from cross-product comparisons, and the answer says so.
Missing fund managers	91.2% filled; **2,084 rows are still NULL**. We built no manager-rename mapping table — without fund inception dates, there is no way to determine which spelling is the current name.
Size of the external documents	Only **3 files, 39 chunks** (ETF terms and metrics, general ETF risk factors, an overview of the National Growth Fund). They do not cover descriptive questions broadly, and questions that need background knowledge absent from the masters are answered as unverifiable.
Provided data vs. task description	The task-description PDF states about 145,000 records across the four datasets with a reference date of 2026-07-11, but the files actually distributed total **53,375 records · reference date 2026-08-21**. This implementation is based on **the files actually provided**; if the evaluation data is updated, re-running `python scripts/build_store.py` is all it takes to pick it up (assuming the same schema).
What we did not measure	**Container startup time** (needs re-measuring after the constituents table was added and the graph rebuilt) · retrieval quality after switching to commercial embeddings · concurrent throughput (not measured, since the organizers specified sequential calls). **Since we did not measure them, we give no figures.**

06-E. Deliverables mapping

Table 14. The organizers' required components ↔ where they are implemented

Task requirement	Implementation
① Structured/unstructured analysis & cleansing + a product-domain-specific Ontology ([Parsing] pdf·ppt → Markdown)	`src/finagent/etl/` (4 structured datasets + constituents) · `scripts/parse_docs.py` (PDF/PPTX → Markdown, with provenance and reference date in the front-matter) · **6** `ontology/*.ttl` files
② Financial-product KnowledgeBase **RDB + Vector + Graph** ([Extraction]·[EntityResolution])	RDB = SQLite **5 views · 215 columns** / Vector = `kb/vector.py` (TF-IDF+LSA, 384 dimensions) / Graph = `kb/graph.py` (1,082,874 triples, inferences included) / Entity Resolution = `kb/entity.py` + `data/enrichment/*.yaml`. Lexical and vector indices hold **75,089 documents**
③ Intent Analysis & Retrieval Engine ([NL2SQL]·[Retrieval])	Pre-analysis (intent · entities · ontology validation) becomes the **retrieval execution plan**, deterministically running only the needed stores in round 0 → holding those results as facts, the LLM finishes aggregation and sorting with `sql_query` (NL2SQL). The key is **not leaving store selection to the whims of the model**
④ Answer Generator (evidence-based · hallucination prevention)	`final_answer` three-state enforcement + `retrieved_context` evidence blocks + **rejection of zero-lookup answers** + unverifiable / clarifying-question policy
⑤ LLM — NCP HyperCLOVA X mandatory	`LLM_PROVIDER=clova` by default; function calling through the CLOVA Studio OpenAI-compatible endpoint (`HCX-005`). Thanks to the adapter design, switching providers costs ≈ 0